Leçon 1 : Introduction aux bases de données

Dr SANGARE Seydou

Table des matières

Objectifs **3**

Introduction **4**

I - De l'organisation classique des fichiers aux bases de données **5**

1. Organisation classique des fichiers 5
2. Caractéristiques de l'organisation classique.....5
3. Limites de l'organisation classique.....6

II - Présentation des Systèmes de Gestion de Base de Données (SGBD) **8**

1. Système d'information 8
2. Notion de base de données.....8
3. Notion de Système de Gestion de Base de Données (SGBD) 9
4. Objectifs d'un SGBD.....9
5. Architecture Client-Serveur.....10
6. Quelques SGBD du marché 11

III - Modèles de SGBD **13**

1. Modèle hiérarchique.....13
2. Modèle réseau.....14
3. Modèle orienté objet 15
4. Modèle déductif 15
5. Modèles NoSQL.....16
6. Modèle relationnel.....17

Conclusion **18**

Objectifs

À l'issue de cette leçon, l'étudiant sera capable de :

- Expliquer pourquoi les fichiers classiques ne suffisent plus pour gérer les données ;
- Définir ce qu'est une base de données et un Système de Gestion de Base de Données (SGBD) ;
- Identifier les principaux objectifs et rôles d'un SGBD ;
- Comprendre l'architecture client-serveur dans le contexte des bases de données ;
- Distinguer les principaux modèles de SGBD ;
- Choisir le modèle de base de données adapté à un besoin concret.

Introduction

1. L'importance des données aujourd'hui

Nous vivons dans un monde où les données sont devenues aussi précieuses que le pétrole. Chaque jour, des milliards de données sont générées : transactions bancaires, publications sur les réseaux sociaux, enregistrements médicaux, notes scolaires, recensements de population, etc.

Imaginez une grande bibliothèque comme celle de l'Université Felix Houphouët Boigny. Sans système d'organisation, retrouver un livre parmi des centaines de milliers serait impossible. Les données numériques, c'est pareil : sans organisation, elles deviennent inutilisables.

2. Comment gérer efficacement les données ?

Prenons l'exemple d'une entreprise qui doit gérer :

- Les actes de naissance et de décès
- Les cartes d'identité nationale
- Les recensements fiscaux
- Les permis de construire
- Les listes électorales

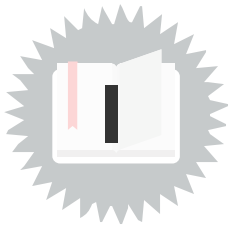
Comment stocker, organiser, retrouver et sécuriser toutes ces informations de manière efficace ?

3. Transition vers les bases de données

La réponse à cette problématique ne se trouve pas dans les classeurs Excel ou les fichiers texte isolés, mais dans les bases de données et les Systèmes de Gestion de Base de Données (SGBD).

Avant d'y arriver, comprenons d'abord pourquoi les méthodes classiques ont montré leurs limites.

De l'organisation classique des fichiers aux bases de données



1. Organisation classique des fichiers

L'organisation classique consiste à stocker les données dans des fichiers indépendants, généralement gérés par des logiciels bureautiques (Excel, Word, fichiers texte).

Exemple: Gestion d'une école secondaire

Imaginons dans un Lycée, l'administration utilise trois fichiers Excel distincts :

Fichier 1 : `eleves.xlsx`

Matricule	Nom	Prénom	Classe	Date naissance
E001	Koné	Amadou	Terminale C	15/03/2008
E002	Bamba	Fatou	Terminale C	22/07/2007

Fichier 2 : `notes.xlsx`

Matricule	Nom	Matière	Note	Trimestre
E001	Koné Amadou	Mathématiques	14	1er
E001	Koné Amadou	Physique	16	1er
E002	Bamba Fatou	Mathématiques	17	1er

Fichier 3 : `paiements.xlsx`

Matricule	Nom élève	Montant payé	Date	Motif
E001	Koné Amadou	50 000 FCFA	05/09/2024	Inscription
E002	Bamba Fatou	50 000 FCFA	06/09/2024	Inscription

Remarque : Le nom de l'élève est répété dans chaque fichier. Si Amadou Koné change de nom (mariage des parents, erreur d'état civil), il faut modifier trois fichiers différents.

2. Caractéristiques de l'organisation classique

Caractéristique	Description	Exemple
Fichiers indépendants	Chaque fichier vit seul, sans lien avec les autres	`eleves.xlsx` ne "connaît" pas `notes.xlsx`

Redondance des données	Mêmes informations répétées dans plusieurs fichiers	Le nom "Koné Amadou" apparaît 3 fois
Gestion manuelle	L'utilisateur doit tout faire à la main (copier, coller, vérifier)	Modifier un nom dans tous les fichiers
Pas de contrôle automatique	Aucun mécanisme empêche les erreurs	On peut saisir une note de 150/20 sans alerte
Accès séquentiel	Pour trouver une information, on doit parcourir tout le fichier	Chercher un élève dans 5 000 lignes
Pas de partage simultané	Deux personnes ne peuvent pas modifier le même fichier en même temps	Le comptable et le proviseur bloquent le fichier
Sécurité limitée	Tout le monde peut ouvrir, modifier ou supprimer	Un stagiaire peut effacer toutes les notes

3. Limites de l'organisation classique

a) Redondance (duplication des données)

La même information est stockée plusieurs fois dans différents endroits.

Exemple : Dans notre lycée, le nom "Koné Amadou" est écrit dans :

- Le fichier des élèves ;
- Le fichier des notes ;
- Le fichier des paiements.

Conséquence : Si on corrige le nom dans un seul fichier, les autres restent faux. La base de données devient incohérente.

b) Incohérence

Exemple :

- Dans `eleves.xlsx` : Koné Amadou est en **Terminale C**
- Dans `notes.xlsx` : Koné Amadou est noté en **Terminale D**
- Dans `paiements.xlsx` : Koné Amadou est en **Première C**

Question : Dans quelle classe est réellement cet élève ? Impossible de le savoir sans vérification manuelle.

c) Difficulté d'accès

Scénario : Le proviseur demande : *"Quels sont les élèves de Terminale C qui n'ont pas encore payé la totalité des frais de scolarité et qui ont une moyenne supérieure à 15 ?"*Avec des fichiers Excel séparés, cette requête nécessite :

1. Ouvrir le fichier des élèves → filtrer Terminale C
2. Ouvrir le fichier des notes → calculer les moyennes
3. Ouvrir le fichier des paiements → vérifier les soldes
4. Comparer manuellement les trois listes

Temps estimé: 2 à 3 heures pour un lycée de 2 000 élèves.

d) Manque de sécurité

Risque	Exemple
Accès non contrôlé	N'importe qui ouvrant l'ordinateur peut voir toutes les notes
Pas de traçabilité	Impossible de savoir qui a modifié quoi et quand
Pas de sauvegarde automatique	Si le disque dur tombe en panne, toutes les données disparaissent
Pas de cryptage	Les données sensibles (numéros de téléphone, adresses) sont en clair

Cas réel : En 2019, une mairie d'Afrique de l'Ouest a perdu l'intégralité de ses registres d'état civil suite à une panne de disque dur. Aucune sauvegarde n'existait.

Présentation des Systèmes de Gestion de Base de Données (SGBD)



1. Système d'information

Un système d'information (SI) est un ensemble organisé de ressources (matérielles, logicielles, humaines, données) permettant de collecter, stocker, traiter et diffuser des informations au sein d'une organisation.

Exemple: Système d'information d'un hôpital

Prenons le Centre Hospitalier Universitaire de Treichville (Abidjan). Son SI comprend :

- **Données** : dossiers médicaux, résultats d'analyses, stocks de médicaments
- **Logiciels** : application de gestion des patients, système de pharmacie
- **Matériel** : serveurs, ordinateurs, scanners de codes-barres
- **Humains** : médecins, infirmiers, administrateurs système
- **Processus** : admission, consultation, hospitalisation, facturation

Un système d'information est comme le système nerveux d'un organisme : il connecte tous les organes et permet à l'organisme de fonctionner de manière coordonnée.

2. Notion de base de données

Une base de données (BD) est un ensemble structuré de données, stockées sur un support informatique, organisé de manière à faciliter la consultation, la mise à jour et la recherche des informations.

Imaginez une bibliothèque bien organisée :

- Les livres sont classés par thème, auteur, date ;
- Chaque livre a une fiche descriptive (titre, auteur, année, résumé) ;
- Le bibliothécaire peut rapidement trouver n'importe quel livre ;
- Les emprunts sont enregistrés et traçables.

Une base de données, c'est la même chose, mais pour des données numériques.

Exemple :

La base de données d'une mairie pour l'état civil contient :

- Tous les actes de naissance (nom, date, lieu, parents)
- Tous les actes de mariage
- Tous les actes de décès
- Les liens entre ces actes (un enfant est lié à ses parents)

3. Notion de Système de Gestion de Base de Données (SGBD)

Un Système de Gestion de Base de Données (SGBD) est un logiciel qui permet de :

- **Créer** une base de données ;
- **Manipuler** les données (ajouter, modifier, supprimer, consulter) ;
- **Contrôler** l'accès et la sécurité ;
- **Assurer** la cohérence et l'intégrité des données.

Le SGBD est comme le bibliothécaire :

- Il connaît l'emplacement de chaque livre ;
- Il vérifie qui peut emprunter quoi ;
- Il s'assure qu'aucun livre ne disparaît mystérieusement ;
- Il peut répondre rapidement à vos questions.

4. Objectifs d'un SGBD

Objectif	Description	Exemple
Indépendance physique	L'utilisateur ne gère pas les fichiers sur disque	Le proviseur ne sait pas où sont stockées les notes physiquement
Indépendance logique	On peut ajouter un champ sans casser les applications existantes	Ajouter "numéro de téléphone parent" sans modifier les bulletins
Manipulation efficace	Recherche en millisecondes même sur des millions de lignes	Trouver un élève parmi 50 000 en moins d'une seconde
Intégrité	Règles automatiques de validation	Refuser une note de 25/20 ou une date de naissance dans le futur
Sécurité	Contrôle fin des accès	Seul le proviseur peut modifier les notes ; les profs ne peuvent que saisir
Partage	Accès simultané multiple	Le comptable et le censeur consultent les données en même temps
Reprise sur panne	Récupération après crash	Après une coupure de courant, aucune donnée n'est perdue
Réduction redondance	Données stockées une seule fois	Le nom de l'élève n'existe qu'à un seul endroit

5. Architecture Client-Serveur

Architecture logicielle ouverte (fidèle aux standards) qui fournit des services distants (BD, messagerie, impression, etc.) à des clients interconnectés via réseaux de manière transparente à l'hétérogénéité des ressources informatiques mises en jeu (ordinateurs, réseaux et logiciels de base).

Un serveur est un dispositif informatique qui offre des services à des clients :

Partage de fichiers ;

Partage de données (BD) ;

Hébergeur web …

Un client est un dispositif informatique qui permet l'envoi de requêtes à un serveur donné, et la réception des réponses. Interface légère gérant de façon conviviale la communication avec le serveur.

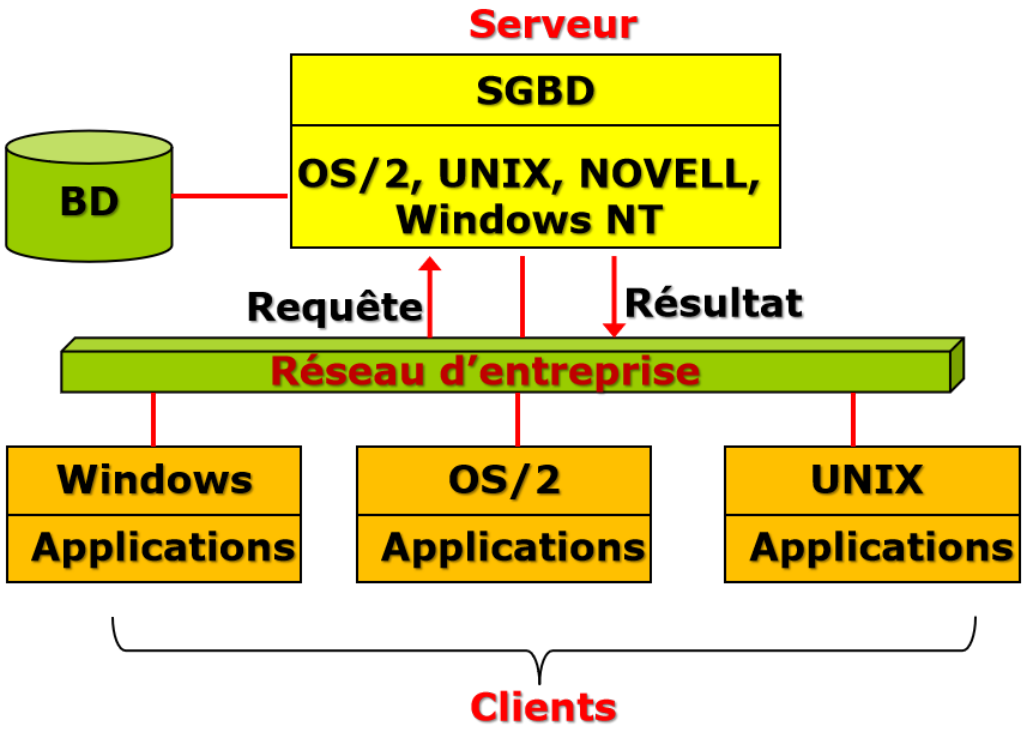


Image 1 Architecture client-serveur

L'architecture client-serveur est le modèle le plus répandu pour les bases de données. Elle sépare les rôles :

Rôle	Description	Analogie
Client	Logiciel qui envoie des requêtes (navigateur, application mobile, logiciel métier)	Le client au restaurant qui passe commande
Serveur	Ordinateur puissant qui héberge la base de données et traite les requêtes	Le chef en cuisine qui prépare le plat
Réseau	Connexion entre client et serveur (Internet, réseau local)	Le serveur qui transporte la commande

Exemple: Gestion d'un hôpital

- **Serveur** : Situé dans la salle informatique de l'hôpital, héberge la base de données des patients
- **Clients** :
    - Ordinateur de l'accueil (enregistrement des admissions)
    - Tablette du médecin (consultation des dossiers)
    - Smartphone de l'infirmier (mise à jour des constantes)
    - Ordinateur de la comptabilité (facturation)

Tous ces clients se connectent simultanément au même serveur, qui gère les accès et garantit la cohérence.

6. Quelques SGBD du marché

SGBD	Type	Éditeur	Gratuit ?	Usage typique	Popularité
MySQL	Relationnel	Oracle Corporation	Oui (open source)	Sites web, applications web	Très élevée
PostgreSQL	Relationnel	Communauté	Oui (open source)	Applications complexes, géospatiales	Très élevée
Oracle Database	Relationnel	Oracle Corporation	Non (coûteux)	Grandes entreprises, banques	Élevée (entreprises)
SQL Server	Relationnel	Microsoft	Non (version gratuite limitée)	Entreprises Microsoft	Élevée
MongoDB	NoSQL (document)	MongoDB Inc.	Oui (open source)	Applications web modernes, big data	Élevée
SQLite	Relationnel (embarqué)	Communauté	Oui	Applications mobiles, IoT	Élevée
MariaDB	Relationnel	Communauté	Oui (open source)	Successeur de MySQL	Élevée
Redis	NoSQL (clé-valeur)	Redis Ltd.	Oui	Cache, sessions temps réel	Moyenne
Cassandra	NoSQL (colonne)	Apache Software Foundation	Oui	Big data, haute disponibilité	Moyenne
Firebase	NoSQL (cloud)	Google	Freemium	Applications mobiles rapides	Moyenne

Dans la suite du cours nous verrons les différents de SGBD.

Comment choisir ?

Situation	SGBD recommandé	Justification
Site web simple (blog, boutique)	MySQL / MariaDB	Simple, rapide, bien documenté
Application géospatiale (cartographie)	PostgreSQL + PostGIS	Excellent support spatial
Grande banque africaine	Oracle / SQL Server	Support technique, sécurité maximale
Application mobile	SQLite / Firebase	Léger, synchronisation cloud
Analyse de millions de tweets	MongoDB / Cassandra	Flexible, horizontal scalable
Système temps réel (trading)	Redis	Extrêmement rapide

Modèles de SGBD

Un modèle de base de données définit la structure logique des données, leurs relations et les contraintes qui régissent leur stockage et leur accès, souvent représentés par des diagrammes.

Les bases de données sont apparues à la fin des années 1960 pour répondre au besoin d'une gestion plus souple et efficace de l'information.

Il existe plusieurs modèles de SGBD, tels que hiérarchique, réseau, relationnel, orienté objet, relationnel-objet, déductif et NoSQL, chacun adapté à des besoins spécifiques.

1. Modèle hiérarchique

Le modèle hiérarchique organise les données sous forme d'arborescence (comme un arbre généalogique), où chaque enregistrement a un et un seul parent.

Principe de fonctionnement

- Les données sont organisées en nœuds (enregistrements)
- Chaque nœud a un lien vers son parent unique
- La navigation se fait de haut en bas (du général au particulier)

Il a surtout été utilisé par les systèmes de gestion d'information d'IBM dans les années 60 et 70, qui ont aujourd'hui majoritairement disparu en raison de certaines inefficacités opérationnelles.

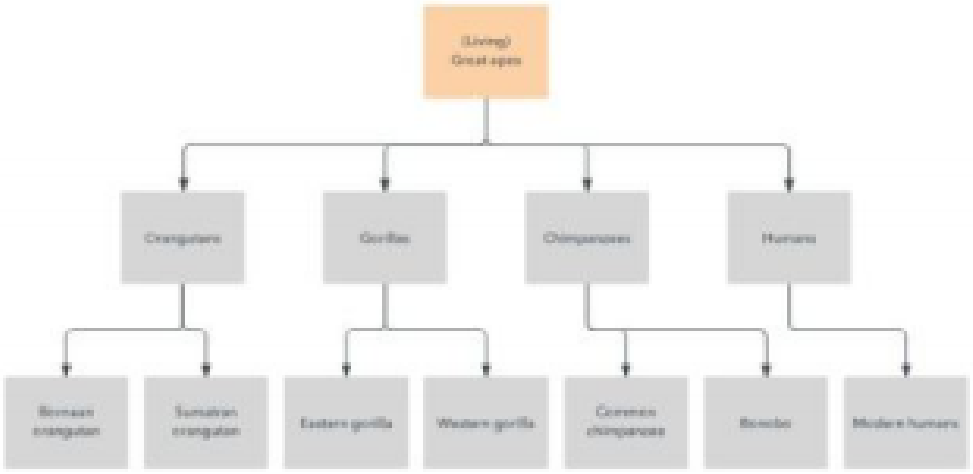


Image 2 Architecture du modèle de SGBD hiérarchique

Avantages

- Structure simple et intuitive
- Très rapide pour les recherches descendantes
- Peut représenter des relations naturellement hiérarchiques

Limites

- Impossible de représenter un enfant avec plusieurs parents (ex: un employé qui travaille dans deux départements)
- Rigidité : difficile de modifier la structure
- Redondance si on veut contourner la limitation

Exemple :

Système de fichiers d'un ordinateur (Windows Explorer, Finder Mac) :

2. Modèle réseau

Le modèle réseau est une extension du modèle hiérarchique qui autorise des relations plusieurs-à-plusieurs entre des enregistrements liés, ce qui implique plusieurs enregistrements parents.

Basé sur la théorie mathématique des ensembles, ce modèle s'articule autour d'ensembles d'enregistrements connexes. Chaque ensemble se compose d'un enregistrement propriétaire (ou enregistrement parent) et d'un ou plusieurs enregistrements membres (ou enfants).

Un enregistrement peut être un membre ou un enfant dans plusieurs ensembles, ce qui permet à ce modèle de traduire des relations complexes.

Son pic de popularité remonte aux années 70, après qu'il a été officiellement défini par la conférence sur les

langages de systèmes de traitement de données (Conference on Data Systems Languages, CODASYL)

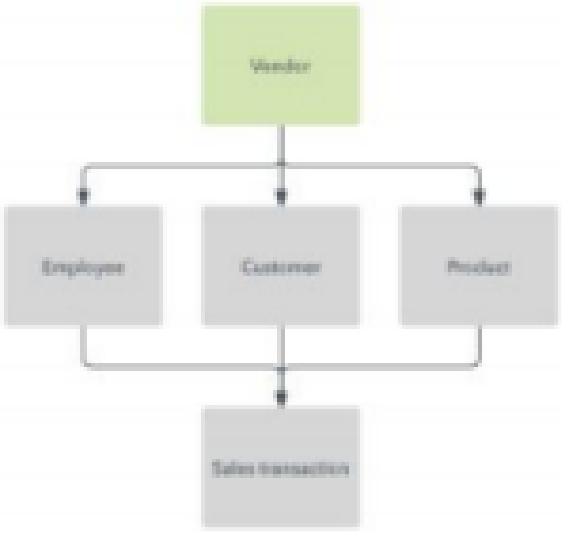


Image 3

Avantages

- Plus flexible que le modèle hiérarchique ;
- Peut représenter des relations complexes (many-to-many) ;
- Navigation rapide grâce aux pointeurs physiques.

Limites

- Structure complexe à comprendre et à modifier ;
- Requêtes difficiles à écrire (navigation par pointeurs) ;
- Dépendance forte entre données et programmes.

3. Modèle orienté objet

Le modèle orienté objet stocke les données sous forme d'objets, comme en programmation orientée objet (Java, C++, Python). Chaque objet a des attributs (données) et des méthodes (comportements).

Principe de fonctionnement

- Les données et les traitements sont regroupés dans des objets ;
- Supporte l'**héritage** (un objet hérite des caractéristiques d'un autre) ;
- Supporte le polymorphisme et l'encapsulation.

Avantages

- Correspondance naturelle avec la programmation orientée objet ;
- Réutilisation du code (héritage) ;
- Modélisation riche des données complexes (multimédia, géométrie).

Limites

- Complexité élevée ;
- Moins mature et moins utilisé que le modèle relationnel ;
- Requêtes moins standardisées.

4. Modèle déductif

Le modèle déductif (ou base de données déductive) combine une base de données relationnelle avec un moteur d'inférence capable de déduire de nouvelles informations à partir de règles logiques.

Principe de fonctionnement

- Stocke des faits (données de base) comme en relationnel ;
- Stocke des règles (logique) sous forme de clauses ;
- Le système peut déduire des conclusions à partir des faits et règles.

Avantages

- Peut répondre à des questions complexes sans stocker toutes les réponses ;
- Représentation élégante des connaissances ;
- Puissance d'expression supérieure au relationnel pur.

Limites

- Complexité algorithmique (peut devenir très lent) ;
- Peu de SGBD commerciaux disponibles ;
- Niche (intelligence artificielle, systèmes experts).

Exemple

Système expert médical dans un hôpital :**Faits** :

- Symptôme(Patient1, fièvre)
- Symptôme(Patient1, maux_de_tête)
- Symptôme(Patient1, fatigue)

Règles :

- SI fièvre ET maux_de_tête ET fatigue ALORS suspect(malaria)
- SI suspect(malaria) ET zone(tropicale) ALORS diagnostic(probable_malaria)

Déduction : Le système déduit automatiquement que Patient1 a probablement la malaria, sans que cette information soit stockée explicitement.

SGBD : Datalog, Prolog (avec extensions BD), LDL, XSB.

5. Modèles NoSQL

NoSQL = **Not Only SQL**. Ce sont des bases de données qui ne suivent pas le modèle relationnel traditionnel. Elles sont conçues pour gérer de très grands volumes de données, avec une grande flexibilité et une haute disponibilité.

Les 4 grandes familles NoSQL

Famille	Structure	Analogie	Exemple SGBD
Clé-valeur	Chaque donnée = clé + valeur	Dictionnaire	Redis, DynamoDB
Document	Données stockées en documents (JSON)	Classeur avec fiches	MongoDB, CouchDB
Colonne	Données organisées par colonnes	Tableau Excel transposé	Cassandra, HBase
Graphe	Données = nœuds + relations	Réseau social	Neo4j, Amazon Neptune

Avantages

- **Scalabilité horizontale** : on ajoute des serveurs pour augmenter la capacité ;
- **Flexibilité du schéma** : pas besoin de définir la structure à l'avance ;
- **Haute disponibilité** : réplication automatique sur plusieurs serveurs ;
- **Performance** : optimisé pour des cas d'usage spécifiques.

Limites

- Moins de garanties d'intégrité que le relationnel ;
- Pas de langage de requête standardisé (contrairement à SQL) ;
- Transactions complexes plus difficiles.

Exemple: Réseau social africain

Imaginez **"AfriConnect"**, un réseau social comme Facebook mais pour l'Afrique :

- 1 milliard d'utilisateurs ;
- Chaque utilisateur a un profil, des amis, des publications, des messages ;
- Les données croissent exponentiellement ;

Pourquoi NoSQL ?

- Le modèle relationnel ne peut pas gérer un milliard de lignes dans une table "amis" ;
- MongoDB stocke chaque profil comme un document JSON flexible ;
- Cassandra gère le fil d'actualité avec sa haute vitesse d'écriture ;
- Neo4j gère les recommandations d'amis ("Amis de mes amis").

SGBD : MongoDB, Cassandra, Redis, Neo4j, Couchbase, DynamoDB.

6. Modèle relationnel

Une base de données relationnelle organise les informations en tables composées de lignes (enregistrements) et de colonnes (attributs), selon le modèle proposé par Edgar F. Codd en 1970.

Ce modèle repose sur la notion mathématique de relation et vise à structurer les données de manière logique, indépendamment des contraintes techniques.

Avantage :

Le modèle relationnel utilise des tables simples à deux dimensions et offre des langages de haut niveau comme SQL, facilitant la manipulation des données même par des non-spécialistes. Il garantit l'indépendance entre les niveaux logique et physique, tout en assurant l'intégrité, la sécurité et l'optimisation des accès aux données.

Enfin, il propose une approche méthodologique rigoureuse, évolutive et adaptée à de nombreuses applications, avec une grande puissance de représentation.

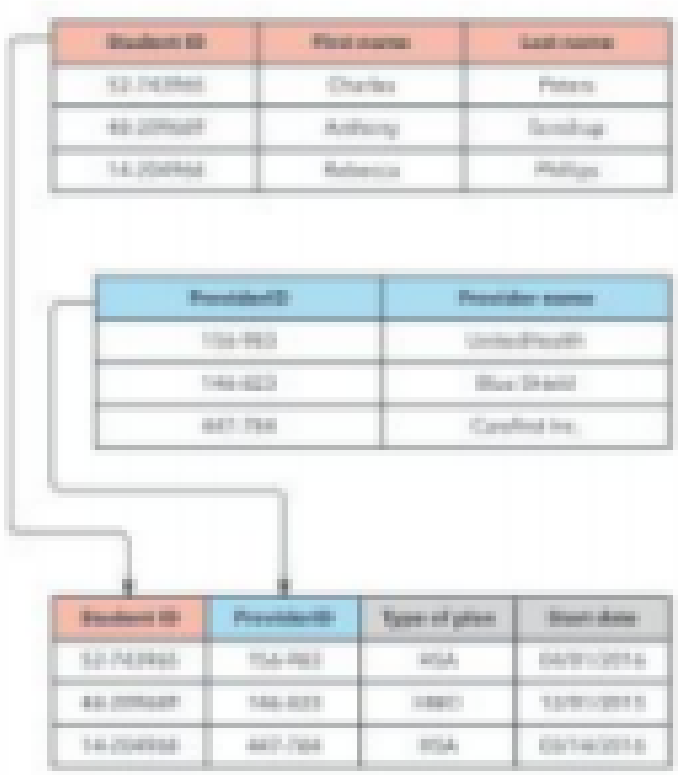


Image 4

Conclusion

Au terme de cette leçon, il apparaît clairement que les méthodes classiques de gestion des fichiers présentent de nombreuses limites, notamment en termes de redondance, d'incohérence, de sécurité et de difficulté d'accès aux données . L'introduction des bases de données et des Systèmes de Gestion de Base de Données (SGBD) constitue ainsi une réponse essentielle pour organiser, sécuriser et exploiter efficacement les données dans un environnement numérique en constante évolution.

Par ailleurs, la diversité des modèles de bases de données (relationnel, NoSQL, orienté objet, etc.) offre aux concepteurs une large palette de solutions adaptées à des besoins variés, allant des applications simples aux systèmes complexes à grande échelle . Le choix du modèle et du SGBD doit donc être guidé par le contexte, les contraintes techniques et les objectifs métier.

Enfin, la maîtrise des concepts fondamentaux présentés dans cette leçon constitue une étape indispensable pour toute personne souhaitant concevoir, développer ou administrer des systèmes d'information performants. Elle ouvre la voie à des apprentissages plus avancés en modélisation, en conception de bases de données et en exploitation des données à grande échelle.